

Projeto 2 – Estufa inteligente na plataforma FIWARE

CCM520 – Internet das Coisas

Departamento de Ciência da Computação

Centro Universitário FEI

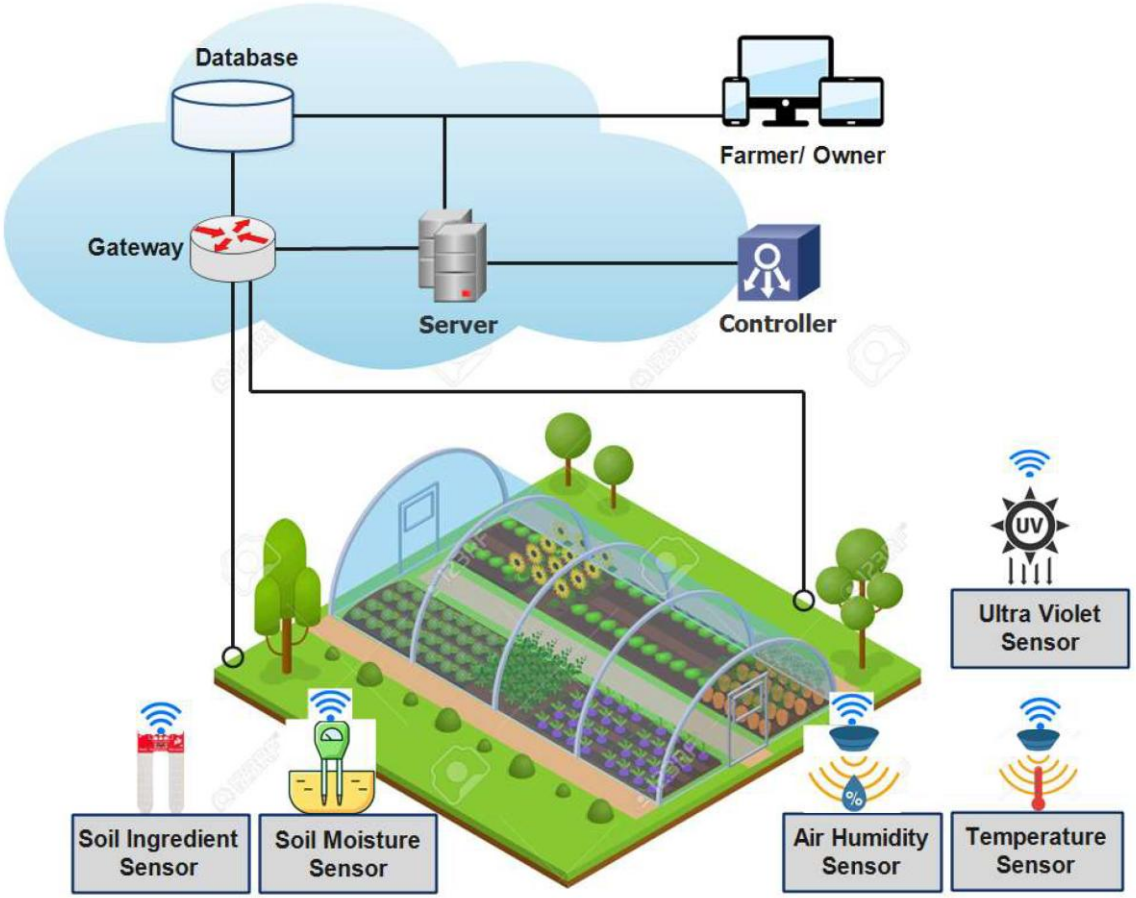
JESUÍTAS BRASIL

1 Descrição do Projeto

Da estufa conectada à estufa gerenciada por contexto.

Aplicação em uma estufa inteligente

- No Projeto 1, a estufa **publicava** suas medidas e **recebia** comandos pelo broker público da HiveMQ.
- Agora os produtores querem que **outras aplicações** usem a estufa: um **painel** com o histórico, **alertas** de solo seco e, no futuro, automação.
- Essas aplicações não devem depender de MQTT, tópicos ou broker: elas falam com o **Orion**, e a estufa conversa com a plataforma pelo **IoT Agent JSON**.



Do Projeto 1 ao Projeto 2

Aspecto	Projeto 1	Projeto 2
Quem manda comandos	você, pelo HiveMQ WebSocket Client	qualquer aplicação, **pelo Orion** (NGSI-v2)
Quem conversa com a ESP32	você	o **IoT Agent JSON**
Tópicos	`/fei/<id>/<grandeza>`	convenção do IoT Agent (medidas, comandos, resultado)
Medidas	um valor por tópico	**um JSON** com todas as medidas
Ativar / Desativar / Atualizar:N	mensagens no tópico `esp32`	comandos `setSending` e `setInterval`
Estado atual e histórico	não havia	**Orion** (atual) e **Cygnus** → **MySQL** → **Grafana**
Broker	`broker.hivemq.com`	o **mesmo** para a ESP32; o IoT Agent o alcança por uma **ponte**

Objetivos de aprendizagem

1. **Integrar dispositivo e plataforma**
   Conectar a ESP32 ao FIWARE (IoT Agent + Orion) por um broker MQTT público e uma ponte MQTT.

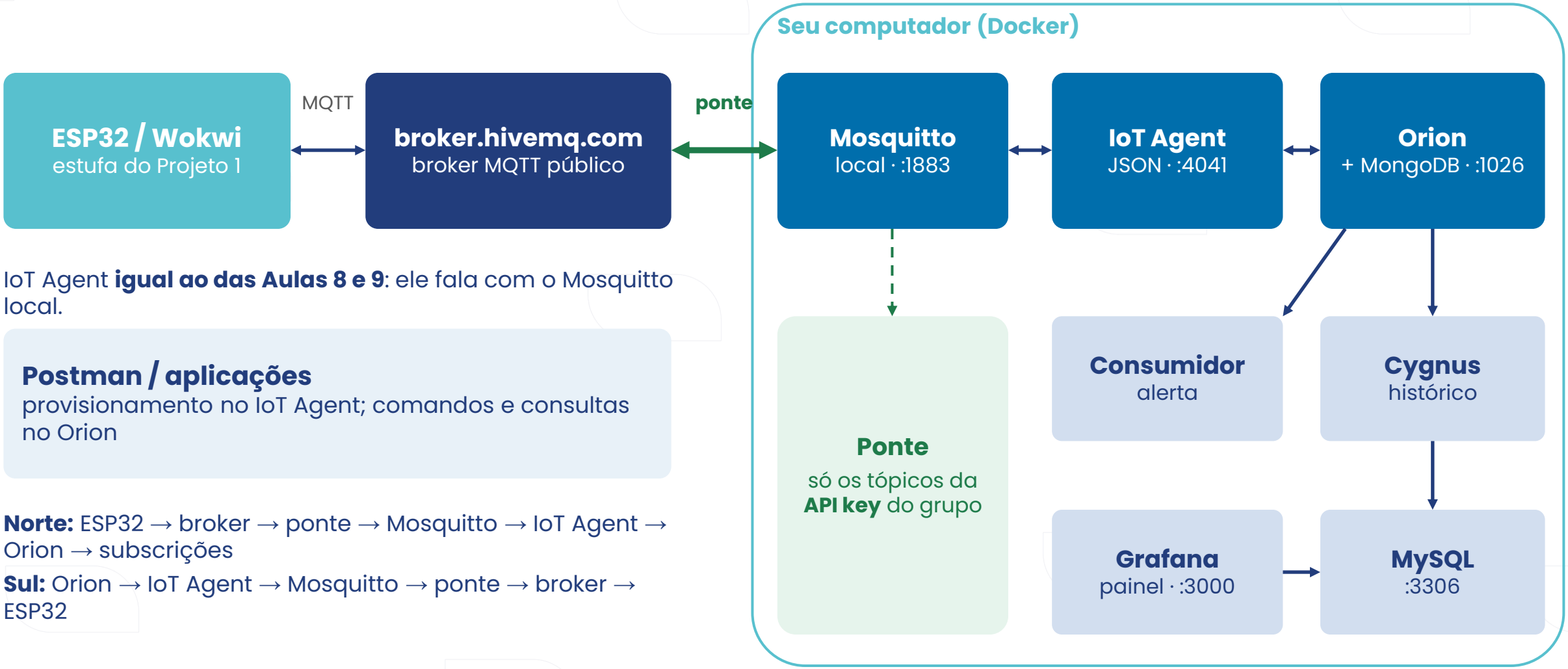
2. **Modelar o contexto**
   Representar a estufa como entidade NGSI, com medidas, comandos e atributos estáticos.

3. **Projetar a comunicação**
   Aplicar a convenção de tópicos e payloads do IoT Agent e o ciclo de vida dos comandos.

4. **Consumir o contexto**
   Usar subscrições para alertas e histórico, e montar um painel no Grafana.

5. **Avaliar riscos**
   Identificar os problemas de um broker público: colisões, assinatura compartilhada e segurança.

Arquitetura do sistema



IoT Agent **igual ao das Aulas 8 e 9**: ele fala com o Mosquitto local.

Postman / aplicações
provisionamento no IoT Agent; comandos e consultas no Orion

Norte: ESP32 → broker → ponte → Mosquitto → IoT Agent → Orion → subscrições

Sul: Orion → IoT Agent → Mosquitto → ponte → broker → ESP32

2. Etapa 1 – Plataforma FIWARE

Orion, IoT Agent e o broker público da HiveMQ.

Requisitos da Etapa 1

- Subir o ambiente do `docker-compose.yml` do Projeto 2: Orion, MongoDB, IoT Agent JSON (**sem alterações**), Mosquitto, consumidor, Cygnus, MySQL e Grafana.
- Configurar uma **ponte** entre o Mosquitto local e o **broker público** da HiveMQ, o mesmo do Projeto 1.
- A ponte só deixa passar os tópicos da **API key do grupo**, que precisa ser **única na turma**.
- Testar a ponte nos dois sentidos e passar nas verificações da pasta *Etapa 1* da coleção Postman.

Por que o broker público?

A estufa (real ou no **Wokwi**) e a plataforma ficam em redes diferentes. O broker público é o ponto de encontro mais simples, e os problemas que ele cria fazem parte do aprendizado.

Atenção

Um broker público **não tem autenticação nem controle de acesso**: qualquer pessoa pode ler e publicar nos seus tópicos. Use-o **apenas** com dados de laboratório.

Cuidados com o broker público

1 API key única

Os tópicos do IoT Agent levam a API key. Duas estufas com a mesma API key misturam medidas e comandos.

2 Client id único

Dois clientes com o mesmo client id derrubam um ao outro: vale para a ESP32 e para a ponte.

3 Assinatura compartilhada

O IoT Agent assina curingas em um grupo `$share` de nome fixo. Ligado direto ao broker público, ele dividiria as medidas com os agentes da turma.

4 Ponte filtrada pela API key

Por isso o IoT Agent fica no Mosquitto local, e a ponte só traz e leva os tópicos da sua API key.

3. Etapa 2 – Modelo de dados e provisionamento

A estufa como uma entidade de contexto.

Entidade Greenhouse: medidas e estáticos

Grandeza	Atributo NGSI	Tipo	Unidade / faixa
Temperatura do ar	`temperature`	Number	°C
Umidade relativa do ar	`relativeHumidity`	Number	%
Umidade do solo	`soilMoisture`	Number	% (0 seco, 100 encharcado), calibrada
Luminosidade	`illuminance`	Number	% (0 escuro, 100 claro), calibrada
Estado do relé	`relayState`	Number	0 ou 1
Ângulo da janela (servo)	`windowAngle`	Number	0 a 180
Intervalo de coleta	`samplingInterval`	Number	2 a 10 s
Nome da estufa (estático)	`name`	Text	definido pelo grupo
Localização (estático)	`location`	geo:json	Point (longitude, latitude)

Entidade `urn:ngsi-ld:Greenhouse:001` · serviço `fei` · caminho `/estufa` · os **object_id** (chaves curtas do JSON) são escolha do grupo.

Comandos: do Projeto 1 ao FIWARE

Projeto 1	Comando NGSI	Valor	Efeito
tópico `/fei/<id>/rele`	`setRelay`	0 ou 1	desliga / liga o relé
tópico `/fei/<id>/servomotor`	`setServo`	0 a 180	posiciona a janela
`Atualizar:N` em `/fei/<id>/esp32`	`setInterval`	2 a 10	intervalo de coleta (s)
`Ativar` / `Desativar`	`setSending`	1 / 0	ativa / desativa o envio das medidas

Regras

Os comandos são enviados **pelo Orion**. A estufa responde **todo** comando com `OK` ou `ERRO: <motivo>`, e valor inválido **nunca** muda o estado. Com `setSending` = 0 as medidas param, mas os comandos continuam funcionando.

Requisitos do provisionamento

- Um **grupo** para a sua API key, tipo `Greenhouse`, recurso `/iot/json`, que **rejeita** dispositivos não provisionados.
- Um **dispositivo** `estufa001` (transporte MQTT) com os **7 atributos**, os **4 comandos** e os **2 estáticos**.
- Os **object_id** documentados no README do repositório e **iguais** aos do firmware.
- Uma **medida de teste** publicada à mão no broker público chega ao Orion antes de qualquer firmware.

Do JSON ao Orion

firmware publica

```
{"<chave>": 24.8, ...}
```

provisionamento traduz

`object_id` → `name`

entidade no Orion

```
temperature = 24.8
```

Chave que não é object_id vira atributo com o próprio nome: um erro de digitação não dá erro, cria um atributo errado.

4 Etapa 3 – Firmware da estufa

O mesmo circuito, agora falando a língua do IoT Agent.

Tópicos do IoT Agent JSON

Sentido	Tópico	Quem publica	Payload
Medidas (norte)	`/json/<apikey>/<device_id>/attrs`	ESP32	um JSON com todas as medidas
Comandos (sul)	`/<apikey>/<device_id>/cmd`	IoT Agent	`{"<comando>": <valor>}`
Resultado (norte)	`/json/<apikey>/<device_id>/cmdexe`	ESP32	`{"<comando>": "OK"}` ou `"ERRO: ..."`

Client id único

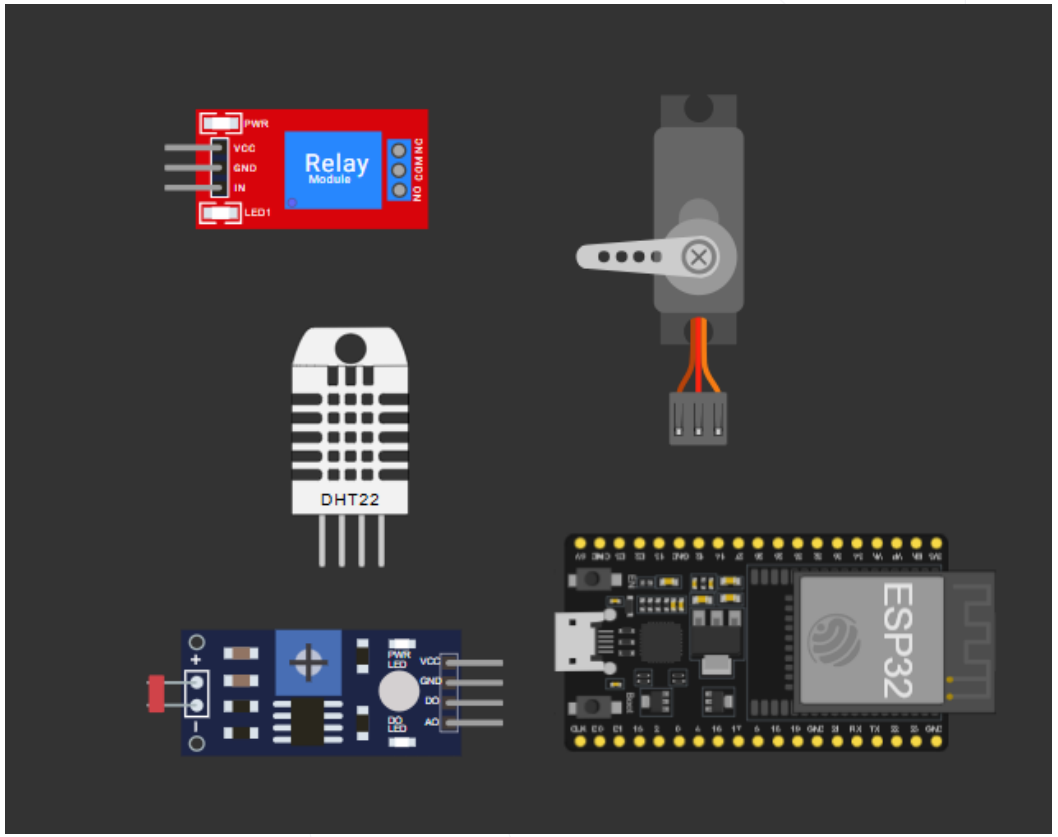
No broker público, use um client id que inclua a API key e o `device_id`. Dois clientes com o mesmo id ficam derrubando um ao outro.

A chave da resposta

A resposta no tópico de resultado precisa usar **exatamente** o nome do comando como chave. Senão o comando fica `PENDING` no Orion.

Requisitos do firmware

- Mesmos pinos e sensores do Projeto 1 (ESP32 real **ou** Wokwi).
- Medidas publicadas em **um JSON**, a cada intervalo (padrão 3 s), com **números** e não textos.
- Os **4 comandos** com validação e resposta `OK` / `ERRO: <motivo>`.
- `setSending` = 0 para as medidas, mas **não** para os comandos.
- Mensagem inválida **nunca** derruba o programa.
- Laço principal **sem** `time.sleep()` e **sem** `wait_msg()`.



Esqueleto: `firmware/main_esqueleto.py` (TODO 1 a 5)

5. Etapa 4 – Consumo do contexto

Alerta, histórico e painel, sem nenhuma linha de MQTT.

Subscrições obrigatórias

1 Histórico → Cygnus

Envia ao **Cygnus** os **7 atributos de medida** das entidades `Greenhouse`.

O Cygnus grava cada atributo como uma linha da tabela `estufa` do banco `fei` (MySQL).

Formato exigido pelo Cygnus: `normalized`.

2 Alerta de solo seco

Notifica `http://consumidor:3000/alerta-estufa` quando `soilMoisture` < **30 %**.

Em `keyValues`, só com `soilMoisture`, `relayState` e `TimeInstant`.

Sem uma notificação a cada medida enquanto o solo estiver seco.

Quem envia as notificações é o **Orion**, de dentro da rede Docker: use os nomes dos serviços, nunca `localhost`.

Painel no Grafana

- Fonte de dados **MySQL** com o usuário **somente leitura** (`grafana`).
- Painel salvo com o título **Estufa inteligente**, contendo:
  - **a)** série temporal de temperatura **e** umidade do ar;
  - **b)** série temporal da umidade do solo;
  - **c)** linha do tempo (*State timeline*) do relé;
  - **d)** *Stat* ou *Gauge* com a luminosidade atual;
  - atualização automática.
- O painel deve **reagir** a um comando enviado pelo Orion.

Fechando o ciclo

`setRelay` = 1 enviado **pelo Orion**:

1. o comando chega à ESP32 pelo broker público;
2. a resposta volta e o Orion mostra `_status` e `_info`;
3. a nova medida mostra `relayState` = 1;
4. o Cygnus grava no MySQL;
5. o Grafana mostra o relé ligado.

Esse é o roteiro da demonstração final.

6. Entregáveis e avaliação

Repositório, demonstração e relatório técnico.

Entregáveis

- **Repositório GitHub** com firmware (sem a senha do Wi-Fi), ambiente, coleção Postman, painel exportado e README (modelo no material de apoio).
- **Coleção Postman** rodando sem falhas no Runner, com a estufa ligada, e o resultado exportado.
- **Relatório técnico** com objetivo, arquitetura, decisões de projeto e as respostas às perguntas **R1 a R8** do guia.
- **Demonstração** ao vivo do fluxo completo, com domínio técnico do caminho dos dados.

```
projeto2-<Nome>-<apikey>/
├── README.md
├── firmware/main.py
├── plataforma/
│   ├── docker-compose.yml
│   └── mosquitto/ mysql/ servicos/
├── postman/
├── grafana/painel-estufa.json
├── relatorio.pdf
```

Regras de integridade acadêmica desta disciplina

	Permitido	Não permitido
Consulta a material de apoio	Apoio com exemplos de trechos de código. Documentações técnicas dos dispositivos. Apresentação das aulas e códigos utilizados em sala.	Cópia direta de projetos da internet.
Uso de Modelos de Linguagem	Solicitação de explicação de trechos de códigos. Verificação de erros de código. Apoio na escrita da documentação do GitHub e relatório técnico.	Pedir para o modelo desenvolver o projeto com base nos requisitos do projeto. Pedir para o modelo criar toda a documentação técnica.
Comunicação entre alunos	Questionamentos sobre os requisitos do projeto. Questionamentos sobre o desenvolvimento do projeto sem apresentação do código completo do projeto. Apoio para verificação de trechos de código e solução de erros.	Cópia de trechos de código desenvolvido por outro sem a devida atribuição. Apresentação completa do código desenvolvido.

Plágio e comissão de ética

- Todos os trabalhos passarão por softwares detectores de plágio. Se algo for identificado, o caso será enviado para avaliação da comissão de ética.
- Softwares de plágio podem indicar que trabalhos feitos com apoio de modelos de linguagem foram copiados.
- Caso a comissão decida pela punição, o trabalho será zerado.

Projeto 2 na prática

A API key, a ponte, os object_id, a calibração e as respostas R1 a R8 são **do grupo**. Na demonstração, cada integrante deve saber explicar o caminho de uma medida e de um comando.

7 Material de Referência

Guia passo a passo, esqueletos e verificações.

Material de apoio (Moodle)

Arquivo	Para que serve
`README.md` e *Guia passo a passo* (PPTX)	O roteiro das 4 etapas, com pontos de verificação
`docker-compose.yml`	Ambiente FIWARE: IoT Agent das Aulas 8 e 9 + Mosquitto local
`mosquitto/mosquitto.conf`	Ponte com o broker público filtrada pela API key (3 TODO)
`firmware/main_esqueleto.py`	Esqueleto do firmware com TODO 1 a 5
`Projeto 2 - Coleção (template)`	Tarefas vazias e verificações automáticas no Postman
`documentacao/Material de apoio.md`	Conceitos: broker público, ponte, *shared subscription*, comandos, segurança
`documentacao/Modelo de README do grupo.md`	Estrutura da documentação do repositório

Referências: material das Aulas 5 a 9 · fiware-iotagent-json.readthedocs.io · fiware-orion.readthedocs.io · hivemq.com/mqtt

Obrigado

CCM520 – Projeto 2

Coordenador da disciplina
Rafael Gomes Alves
ralves@fei.edu.br

JESUÍTAS BRASIL